Spécifications : Système d'Annotations UML

Bibliothèque de Génération Java vers PlantUML

Documentation Architecturale

10 août 2026

Table des matières

1 Introduction **1**
- 1.1 Comment le code est lu . . . 1
- 1.2 Périmètre . . . 1
- 1.3 Principe directeur : la sobriété de l'API . . . 1

2 Règles générales et conventions **2**
- 2.1 Convention de nommage . . . 2
- 2.2 Comment un membre est écrit . . . 2
- 2.3 Un seul énuméré de direction . . . 3
- 2.4 Annotations répétables . . . 3
- 2.5 Cibles autorisées . . . 3
- 2.6 Résolution des conflits . . . 3

3 Le Déclencheur et le Seuil : `@UMLClass` **4**
- 3.1 Principe . . . 4
- 3.2 Paramètres . . . 4
- 3.3 Le paramètre de stratégie . . . 4
- 3.4 Le bruit : rien n'est masqué sans demande . . . 5
- 3.5 Exemple . . . 5

4 La Dérogation d'Exclusion : `@UMLExclude` **5**
- 4.1 Principe . . . 5
- 4.2 L'exclusion se déclare là où le membre est déclaré . . . 5
- 4.3 Exemple . . . 5

5 La Dérogation d'Inclusion : `@UMLInclude` **6**
- 5.1 Principe . . . 6
- 5.2 Paramètres . . . 6
- 5.3 Le renommage d'un membre . . . 6
- 5.4 La notation attribut d'un accesseur . . . 6
- 5.5 Exemple . . . 6

6 La Précision Métier : `@UMLAssociation` **7**
- 6.1 Principe . . . 7
- 6.2 Paramètres . . . 7
- 6.3 Exemple . . . 7

6.4 Les deux notations d'une association 8
6.5 Deux attributs qui se font face ne sont qu'une association 8

7 La Classe d'Association : `@UMLAssociationClass` 9
7.1 Principe 9
7.2 Paramètres 9
7.3 Exemple 9

8 La Multiplicité d'une propriété : `@UMLMultiplicity` 10
8.1 Principe 10
8.2 Pourquoi c'est utile ? 10
8.3 Paramètres 10
8.4 Ce qu'elle ne déduit pas, et ce que le projet peut traduire 10
8.5 Combinaison avec les autres annotations 10

9 Les Éléments Dérivés : `@UMLDerived` 10
9.1 Principe 10
9.2 Exemple 11

10 Les Contraintes Métier : `@UMLConstraint` 11
10.1 Principe 11
10.2 Paramètres 11
10.3 Où la contrainte est écrite 11
10.4 Exemple 11

11 Les Liens Transversaux : `@UMLDependency` 12
11.1 Principe 12
11.2 Paramètres 12
11.3 Exemple 12

12 La Documentation Visuelle : `@UMLNote` 12
12.1 Principe 12
12.2 Paramètres 12
12.3 Ce à quoi une note peut s'accrocher 12
12.4 Exemple 13

13 Le Regroupement Logique : `@UMLPackage` 13
13.1 Principe 13
13.2 Paramètres 13
13.3 Exemple 13

14 Les règles du projet : le fichier `umlg.rules` 13
14.1 Principe 13
14.2 Le partage avec les annotations 13
14.3 Forme du fichier 14
14.4 Le vocabulaire complet 14
14.5 Chaque mot-clé par l'exemple 15
14.6 Le fichier minimal 17
14.7 Un fichier complet 17
14.8 Ce qui est refusé 17
14.9 Ce qui décide d'une règle future 18

15 L'apparence du diagramme : `UmlStyle` **18**
15.1 Principe . . . 18
15.2 Ce qui échappe au choix . . . 19

16 Les vues : plusieurs diagrammes pour un projet **19**
16.1 Principe . . . 19
16.2 Déclarer une vue . . . 19
16.3 L'ordre d'écriture est l'ordre de lecture . . . 20
16.4 Les vues calculées . . . 20
16.5 Les liens qui sortent d'une vue . . . 20
16.6 Passer d'une vue à l'autre . . . 21
16.7 Quand le dessin devient illisible . . . 21
16.8 Ce que la génération dit d'elle-même . . . 21
16.9 Ce que la génération signale . . . 21
16.10 Ce que devient la génération précédente . . . 22
16.11 Lancer la génération . . . 23

17 La vérification à la compilation **24**
17.1 Principe . . . 24
17.2 Deux gravités . . . 24
17.3 Ce qui arrête la compilation . . . 24
17.4 Ce qui produit un avertissement . . . 25
17.5 Faire taire les avertissements . . . 26
17.6 Donner le fichier de règles au compilateur . . . 26

18 Choix de conception : ce qui est hors périmètre **26**
18.1 Fonctionnalités écartées . . . 26
18.2 Ce qui relève de l'analyse automatique . . . 27
18.3 Ce qui ne peut pas être déduit . . . 27

1 Introduction

Ce document référence le système d'annotations Java conçu pour la génération automatique de diagrammes PlantUML. L'outil gère nativement les concepts standards — héritage via `extends`, implémentation via `implements`, classes abstraites, énumérations, *records* ; les annotations ci-dessous viennent enrichir et paramétrer cette analyse.

1.1 Comment le code est lu

La bibliothèque travaille **par réflexion sur les classes compilées** : on lui désigne un répertoire de `.class` (typiquement `target/classes`), et non un répertoire de sources. Conséquence directe : **ce que la compilation efface est perdu pour le générateur** — les valeurs d'initialisation des attributs, et le corps des méthodes, d'où une dépendance qui ne se lit que dans un appel devra être déclarée.

Ce que la compilation n'efface pas. L'effacement de type (*type erasure*) est souvent invoqué à tort : il vaut de ce qui vit à l'exécution, non des **signatures déclarées**. Le compilateur écrit dans le `.class` un attribut `Signature` qui conserve le type tel qu'il a été écrit ; la réflexion l'expose, et la bibliothèque s'en sert : un `List<Commande>` dit à lui seul quelle classe l'association vise. Le principe retenu reste néanmoins : **rien n'est deviné, tout est déclaré**

— une déduction automatique serait de toute façon incapable de trancher entre `0..*` et `1..*`, qui relèvent du métier.

Une seconde lecture, indépendante, a lieu **à la compilation** : la bibliothèque embarque un processeur d'annotations qui vérifie ce que le développeur écrit, là où il l'écrit (section 17). Une dernière exception dépend du projet analysé et non de la bibliothèque : les **noms des paramètres de méthode** ne sont écrits dans les `.class` que si celui-ci est compilé avec `-parameters`. Le générateur les affiche lorsqu'ils existent, se limite aux types sinon, et le **signale** (section 2.2).

1.2 Périmètre

La bibliothèque cible exclusivement le **diagramme de classes**. Les autres diagrammes de PlantUML (séquence, composants, états, cas d'utilisation) sont hors périmètre.

1.3 Principe directeur : la sobriété de l'API

La valeur de la bibliothèque ne se mesure pas au nombre de fonctionnalités PlantUML rendues accessibles, mais au **nombre d'annotations qu'un développeur doit écrire pour obtenir un diagramme correct**. Dans le cas nominal, `@UMLClass` doit suffire. Un jeu d'annotations trop riche déplacerait le problème au lieu de le résoudre : on écrirait alors davantage de spécification UML que de Java. Les directives de rendu purement esthétiques sont donc écartées (section 18).

Il en découle **trois endroits**, et trois seulement, où quelque chose se décide :

- une **annotation** dit une chose sur *cet* élément-ci ;
- le fichier `umlg.rules` dit une chose qui vaut pour tout le projet, ou pour une vue (section 14) ;
- l'**appel du générateur** dit ce qui ne touche pas au modèle — l'apparence, la destination (sections 15 et 16.10).

Le partage est **étanche** : aucune règle ne refait ce qu'une annotation sait faire, et réciproquement. La question « où dois-je écrire ceci ? » a donc toujours une seule réponse, donnée par un critère d'une ligne (section 14.2).

2 Règles générales et conventions

2.1 Convention de nommage

Tout paramètre désignant une classe visée par un lien s'appelle `target` (et `source` pour l'extrémité de départ), uniformément sur `@UMLAssociation`, `@UMLAssociationClass` et `@UMLDependency`.

2.2 Comment un membre est écrit

Le générateur emploie la notation d'UML, `nom : Type`, et non l'ordre de déclaration de Java : c'est l'ordre dans lequel un diagramme se lit, où l'on cherche une propriété par son nom. Les deux-points sont **entourés d'espaces**, comme les écrit la norme UML.

```
class "Vol" {
  - departureTime : Date
  - escales : Aeroport [0..*]
  --
  + reporter(nouvelleHeure : String, minutes : int) : void
  + {static} /compter(code : String) : int
}
```

L'ordre des morceaux est fixe. Pour un attribut : la visibilité, `{static}` s'il y a lieu, le nom (précédé de sa barre oblique s'il est dérivé), le type, la multiplicité entre crochets, puis les contraintes entre accolades. Pour une méthode : la visibilité, `{static}`, le nom, les paramètres entre parenthèses, le type de retour, puis sa multiplicité.

Le type de retour. Il est écrit par défaut, `void` compris. UML le tient pourtant pour facultatif, et l'omettre veut dire « ne rend rien ». Le choix se déclare une fois pour tout le projet, par la règle `omit return void`.

Les noms des paramètres. Ils ne sont affichés que si le projet a été compilé avec `-parameters` ; sans elle, la réflexion ne rend que `arg0`, `arg1`…, que le générateur **n'écrit jamais**. La signature se réduit alors à `reporter(String, int): void`. Pour Maven :

```
<configuration>
    <parameters>true</parameters>
</configuration>
```

La dégradation est **signalée** : quand au moins une méthode dessinée a des paramètres et qu'aucun n'a de nom, un avertissement nomme l'option manquante. La condition est stricte pour ne jamais accuser à tort — un seul nom présent suffit à se taire, et un projet qui les a masqués par `omit parameter names` les a tus volontairement. La forme courte peut aussi être un **choix** : les signatures complètes font des boîtes larges.

L'ordre des membres. Par défaut, **ordre alphabétique**. Ce n'est pas un choix esthétique mais une exigence de reproductibilité : la réflexion rend les membres dans un ordre non spécifié, qui varie d'une exécution à l'autre. La règle `order declaration` rétablit l'ordre du source en relisant les tables de membres du `.class`, qui ne change pas entre deux lectures. Les littéraux d'une énumération font exception : leur ordre est celui des ordinaux.

Le type devant une multiplicité. Dès qu'une multiplicité est écrite, le type qui la précède est celui de l'**élément**, jamais celui du conteneur : un `Set<String>` marqué `[0..*]` s'écrit `alias : String [0..*]` — « zéro ou plusieurs ensembles » serait faux. Une `Map`, ayant deux paramètres, s'écrit en entier.

La multiplicité. Elle n'est écrite que là où elle est déclarée : par `targetCardinality` sous `notation = ATTRIBUTE` (section 6.4), ou par `@UMLMultiplicity` (section 8). Une multiplicité absente vaut *exactement un*, et ne s'écrit donc pas.

2.3 Un seul énuméré de direction

`UMLDirection` (`NONE`, `UP`, `DOWN`, `LEFT`, `RIGHT`) est utilisé partout où une orientation est demandée. `NONE` est le défaut : PlantUML place l'élément lui-même, ce qui convient dans la quasi-totalité des cas. Pour une note, `UP` et `DOWN` placent la bulle au-dessus et en dessous.

2.4 Annotations répétables

`@UMLDependency`, `@UMLNote` et `@UMLConstraint` sont `@Repeatable`. Les autres (`@UMLClass`, `@UMLAssociation`, `@UMLAssociationClass`, `@UMLMultiplicity`, `@UMLPackage`, `@UMLDerived`, `@UMLInclude`, `@UMLExclude`) sont uniques sur leur cible.

```
@UMLDependency(target = ServiceBancaire.class, label = "facture via")
@UMLDependency(target = Journal.class, label = "trace")
@UMLClass
public class Panier { }
```

2.5 Cibles autorisées

Cible	Annotations acceptées
Type (classe, interface, énumération)	`@UMLClass`, `@UMLPackage`, `@UMLAssociationClass`, `@UMLDependency`, `@UMLNote`, `@UMLConstraint`
Attribut	`@UMLInclude`, `@UMLExclude`, `@UMLAssociation`, `@UMLMultiplicity`, `@UMLDerived`, `@UMLConstraint`, `@UMLNote`
Méthode	`@UMLInclude`, `@UMLExclude`, `@UMLMultiplicity`, `@UMLDerived`, `@UMLNote`

Une annotation posée hors de ces cibles est rejetée à la compilation.

2.6 Résolution des conflits

— `@UMLInclude` + `@UMLExclude` : **erreur**. Arbitrer en silence reviendrait à choisir à la place de l'auteur, dans un code qui se contredit.

— `@UMLInclude(name)` + `targetRole` sous `ATTRIBUTE` : **erreur**. En notation attribut, le rôle cible *est* le nom de la ligne.

— Classe visée par un lien sans `@UMLClass` : **erreur** sur le `target` fautif, une cible étant toujours écrite exprès. *Exception* : les héritages, lus dans la signature et non déclarés, sont **ignorés silencieusement** — tout hérite d'`Object`. Aucune classe n'est créée implicitement, ce qui garantit que le diagramme ne déborde jamais du périmètre choisi.

— `@UMLConstraint` sur un attribut porteur de `@UMLAssociation` : la contrainte s'applique à l'**association** (usage majoritaire : `{subset}`, `{ordered}`), et est écrite là où l'association est dessinée. Sans association, elle s'applique à l'attribut.

— `@UMLDerived` sur un tel attribut : c'est l'association qui est dérivée. Sous `ARROW`, la barre se pose devant le **rôle cible** — sans `targetRole`, rien ne peut la porter et le compilateur avertit ; sous `ATTRIBUTE`, elle précède le nom de la ligne.

— `@UMLExclude` sur un tel attribut : ce qui disparaît est ce que l'association demandait — le trait sous `ARROW`, la ligne sous `ATTRIBUTE`. Avertissement dans les deux cas.

— Stratégie de visibilité + `@UMLAssociation` : une association est **toujours** dessinée, dans les deux notations — une demande explicite l'emporte sur un défaut.

— Même `@UMLPackage(name)`, deux couleurs : la première rencontrée est retenue, avertissement.

3 Le Déclencheur et le Seuil : `@UMLClass`

3.1 Principe

Placée sur une classe, interface ou énumération, elle indique que l'entité doit figurer sur le diagramme et fixe un seuil de visibilité par défaut pour ses membres. Sans elle, le générateur tenterait de dessiner tout le projet, classes utilitaires et dépendances comprises : elle choisit le périmètre.

3.2 Paramètres

— `name` (String) : surcharge le nom affiché.

— `color` (String) : couleur de fond — code hexadécimal (`"#DDEEFF"`, `"#ABC"`, `"#FF000080"` avec transparence) ou nom PlantUML (`"LightBlue"`). Le catalogue s'obtient par `UMLGenerator.couleursConnues()`, et le message de refus y renvoie. La question est posée au catalogue réel de PlantUML, et à lui seul : une grammaire écrite en doublon divergeait, refusant `"#FF000080"` que le moteur accepte.

— `stereotype` (String) : étiquette métier (ex. `"Service"`). Les guillemets « » sont posés par le générateur ; les écrire soi-même les afficherait en double, ce dont le compilateur avertit.
— `strategy` (Enum) : le seuil de visibilité, ci-dessous.

3.3 Le paramètre de stratégie

— `PROJECT` **(défaut)** : la classe s'en remet à la ligne `strategy` du fichier `umlg.rules`, ou à `NONE` si le projet n'en déclare aucune. C'est ce qui permet d'écrire la stratégie une fois pour tout le projet.
— `NONE` : coquille vide, sauf les membres explicitement `@UMLInclude`. Écrite sur une classe, elle l'emporte sur le réglage du projet.
— `PUBLIC` / `PROTECTED` / `PACKAGE` / `PRIVATE` : respectivement les membres `public` ; `public` et `protected` ; `public` et sans modificateur ; tous.

Les composants d'un record échappent à la stratégie. `record Mnemonic(String keysymMnemonic)` ne dit rien d'autre que ce composant : il **est** la définition du record. Les composants se dessinent donc quelle que soit la stratégie, `NONE` comprise. `@UMLExclude` garde le dernier mot ; un champ *ajouté* au record (forcément statique) reste un membre ordinaire.

3.4 Le bruit : rien n'est masqué sans demande

Une classe réelle a plus de membres qu'un diagramme lisible n'en supporte, et certains n'apprennent rien. La bibliothèque **n'en masque aucun de sa propre initiative**, et ne cherche même pas à les reconnaître : le tri se déclare dans `umlg.rules`, en nommant les membres à écarter (section 14).

Ce n'est pas une commodité : le jugement « ceci n'apprend rien » est bon dans beaucoup de projets et mauvais dans d'autres — sur un code immuable écrit à la manière de Guava, il n'y a pas un accesseur à écarter, et ce qui encombre est ailleurs. Seul l'auteur du projet le sait, et c'est lui qui l'écrit. Une exception, et une seule : ce que le compilateur a fabriqué et que le développeur n'a jamais écrit — attributs synthétiques, référence à la classe englobante — reste écarté sans qu'on le demande. Les constructeurs, eux, ne sont jamais lus : aucune annotation ne peut en faire apparaître un.

3.5 Exemple

Ci-dessous, le projet a écrit `hide get*` dans son `umlg.rules` ; sans cette ligne, `getPseudo()` paraîtrait comme n'importe quelle autre méthode publique.

```
// Affiche les membres que la strategie autorise, moins ce que le
// fichier de regles ecarte.
@UMLClass(stereotype = "Entity", strategy = UMLVisibility.PACKAGE)
public class Utilisateur {
    public String pseudo;       // Sera affiche
    String email;               // Sera affiche (package-private)
    protected int age;          // Sera ignore (hors strategie)
    private String motDePasse;  // Sera ignore (hors strategie)

    public Utilisateur() {}     // Sera ignore (jamais de constructeur)
    public String getPseudo() {}// Sera ignore (motif "hide get*")

    public void valider() {}    // Sera affiche (methode metier publique)
}
```

4 La Dérogation d'Exclusion : `@UMLExclude`

4.1 Principe

Exception négative : force la dissimulation d'un attribut ou d'une méthode que la `strategy` autorisait à paraître. Utile sous `strategy = PRIVATE`, où les variables techniques (loggers, constantes internes) alourdissent le diagramme sans valeur architecturale. **Aucun paramètre.**

4.2 L'exclusion se déclare là où le membre est déclaré

Posée sur la déclaration d'une méthode — dans une interface, ou dans une classe parente — , l'annotation vaut pour **chacune de ses redéfinitions**. Le besoin vient d'un projet réel : une méthode `asString()` déclarée par une interface et réalisée par trois records devait être exclue trois fois. Une méthode statique ou privée, qui ne redéfinit rien, ne regarde qu'elle-même.

4.3 Exemple

```
@UMLClass(strategy = UMLVisibility.PRIVATE)
public class ConnexionBase {
    private String url; // Affiche car strategie globale sur PRIVATE

    @UMLExclude
    private Logger log; // Ignore explicitement pour alleger le graphe
}
```

5 La Dérogation d'Inclusion : `@UMLInclude`

5.1 Principe

Exception positive : force l'apparition d'un membre que son niveau d'encapsulation aurait masqué, et peut de surcroît le **renommer**. Elle permet de cibler précisément les éléments internes cruciaux sans exposer le reste.

5.2 Paramètres

— `name` (String) : le nom sous lequel le membre paraît, à la place du sien. La chaîne vide — le défaut — veut dire « le nom du code ».

— `asAttribute` (boolean, `false` par défaut) : écrit la méthode en **notation attribut** — `xKeycodeByXKeyName : Map<String, Short>`, sans parenthèses, dans le compartiment des attributs.

5.3 Le renommage d'un membre

Le besoin vient des diagrammes de référence, qui nomment souvent les choses plus longuement que le code : un attribut `codeByName` qu'un README appelle `xKeycodeByXKeyName`. Sans renommage, le diagramme engendré ne peut jamais rejoindre le diagramme voulu. Le renommage est porté par l'annotation, et non par une règle : c'est une décision sur *un* élément. Les notes suivent le nom affiché. Trois écarts sont vérifiés : un nom fait d'espaces est une **erreur** ; `name` et `targetRole` sur la même ligne sont une **erreur** ; un nom identique à celui du code est un **avertissement**. Un `@UMLInclude` qui renomme n'est jamais signalé « sans effet » : renommer *est* un effet.

5.4 La notation attribut d'un accesseur

Un accesseur sans paramètre expose une **propriété**, et les diagrammes qui montrent une structure l'écrivent comme telle. `asAttribute = true` opère la conversion : la ligne quitte le

compartiment des méthodes pour celui des attributs, garde son soulignement si la méthode est statique, sa barre oblique si elle est dérivée, sa multiplicité si elle en porte une.

La décision est **individuelle** parce que le même diagramme mélange les deux formes : une fabrique n'est pas une propriété, et une règle globale les aurait converties avec le reste. La notation exige une méthode **sans paramètre** qui **rend quelque chose** : les trois écarts — des paramètres, un retour `void`, l'annotation posée sur un attribut — sont des **erreurs** de compilation.

5.5 Exemple

```
@UMLClass(strategy = UMLVisibility.PUBLIC)
public class MoteurDeRecherche {

    public void chercher() {} // Affiche car public

    private int compteur; // Ignore car private

    @UMLInclude
    private void algorithmeSecret() {} // Force l'affichage grace a l'
   annotation

    @UMLInclude(name = "xKeycodeByXKeyName")
    public Map<String, Short> codeByName() { ... }
    // Affiche : xKeycodeByXKeyName() : Map<String, Short>
}
```

6 La Précision Métier : `@UMLAssociation`

6.1 Principe

Précise la sémantique UML d'une relation portée par un attribut : type de lien, cardinalités, rôles, classe ciblée. Le code `List<Commande> commandes` indique qu'il y a plusieurs commandes ; seul le métier sait si c'est `0..*` ou `1..*`, et s'il s'agit d'une association ou d'une composition forte.

6.2 Paramètres

- `target` (Class) : la classe cible. **Facultatif** : un attribut déclaré `List<Commande>` nomme déjà sa cible, et la bibliothèque la lit (section 1.1). Il reste nécessaire là où le type ne dit rien — une collection écrite sans paramètre, une `Map` dont rien ne dirait laquelle des deux moitiés est visée, un lien vers une classe absente du type.
- `notation` (Enum) : `ARROW` (défaut) ou `ATTRIBUTE` — la **forme** du dessin (section 6.4).
- `type` (Enum) : `ASSOCIATION`, `COMPOSITION`, `AGGREGATION`.
- `sourceCardinality`, `targetCardinality` (String) : multiplicités (ex. `"1"`, `"0..1"`, `"0..*"`).
- `sourceRole`, `targetRole` (String) : noms des rôles aux extrémités.
- `label` (String) : texte sur la flèche (ex. `"possede"`).
- `direction` (Enum) : `UMLDirection`. Contraint le placement de *ce* trait ; c'est le seul levier de mise en page qui vive sur un élément, et il ne peut pas se périmer — si l'attribut disparaît, la directive part avec lui.

`{ordered}` n'est pas un paramètre. Un `isOrdered` figurait ici, qui produisait exactement ce qu'écrit `@UMLConstraint(rule = "ordered")` : mêmes accolades, même place, indiscernable sur le dessin. L'ordre se déclare donc comme n'importe quelle autre règle formelle.

```java
@UMLConstraint(rule = "ordered")
@UMLAssociation(target = Commande.class, targetCardinality = "0..*")
private List<Commande> commandes;   // "achats 0..* {ordered}"
```

L'association qualifiée est hors périmètre. PlantUML n'a pas cette notation ; le `qualifier` qui la déclarait était rendu entre crochets sur l'extrémité, ce qu'aucun lecteur ne reconnaît sans explication (section 18).

6.3 Exemple

```java
public class Utilisateur {
    @UMLConstraint(rule = "ordered")
    @UMLAssociation(
        target = Commande.class,
        type = UMLRelationType.COMPOSITION,
        sourceCardinality = "1",
        targetCardinality = "0..*",
        targetRole = "achats",
        label = "passe"
    )
    private List<Commande> commandes;
}
```

6.4 Les deux notations d'une association

UML offre deux façons de dessiner une même propriété, et n'en autorise **qu'une à la fois** : un trait entre deux boîtes, ou une ligne dans la boîte de départ. Les deux disent la même chose ; le choix relève de la lisibilité. La bibliothèque tient la règle du non-doublon — sous `ARROW` l'attribut est retiré de la boîte au profit du trait, sous `ATTRIBUTE` il y reste et aucun trait n'est tracé.

Valeur	Ce qui est dessiné	Quand la choisir
`ARROW` (défaut)	Un trait décoré à ses extrémités : `Client "1" *-- "achats 0..*" Commande`	Les associations qui structurent le domaine, celles que le lecteur doit voir en premier.
`ATTRIBUTE`	Une ligne du compartiment des attributs : `- commandes: Commande [0..*]`	Quand une classe en cite dix autres, ou quand la cible n'a rien d'intéressant à montrer : la toile d'araignée disparaît.

```java
@UMLClass
public class Client {
    // Un trait vers Commande (comportement par defaut).
    @UMLAssociation(target = Commande.class, targetCardinality = "1")
    private Commande courante;

    // Une ligne dans la boite : "- archivees: Commande [0..*]"
    @UMLAssociation(target = Commande.class, targetCardinality = "0..*",
                    notation = UMLNotation.ATTRIBUTE)
    private List<Commande> archivees;
}
```

Ce que la notation `ATTRIBUTE` retient. Une ligne de boîte ne porte qu'un nom, un type et une multiplicité. Trois paramètres y trouvent donc leur place : `target` donne le **type** affiché (la

classe visée, et non `List`) ; `targetCardinality` la **multiplicité** entre crochets ; `targetRole` le **nom** de la propriété, s'il est renseigné — à défaut, le nom de l'attribut Java. S'y ajoutent, comme pour n'importe quelle ligne, la visibilité et `{static}`, la barre oblique de `@UMLDerived`, puis les `@UMLConstraint` dans l'ordre où elles sont écrites.

```
@UMLConstraint(rule = "ordered")
@UMLConstraint(rule = "non vide")
@UMLDerived
@UMLAssociation(target = Commande.class, targetCardinality = "1..*",
                notation = UMLNotation.ATTRIBUTE)
protected List<Commande> achats;
// Ligne produite : # /achats: Commande [1..*] {ordered} {non vide}
```

Ce qu'elle laisse de côté. Les paramètres qui décorent un *trait* n'ont nulle part où s'écrire : `type`, `sourceCardinality`, `sourceRole`, `label`, `direction`. Le diagramme reste valide, mais l'auteur a écrit des valeurs pour un effet qu'il n'aura pas : le compilateur les lui nomme.

Ce qu'elle ne relâche pas. La classe visée doit porter `@UMLClass` dans les deux notations, et la grammaire des multiplicités reste la même.

6.5 Deux attributs qui se font face ne sont qu'une association

Un modèle métier est plein d'attributs réciproques : `Eleve` cite `Cours`, et `Cours` cite `Eleve`. Chaque classe déclare son `@UMLAssociation`, et chacune a raison — elle décrit son propre code. UML, lui, n'y voit qu'**une** association, dont chaque extrémité est nommée *par la classe d'en face*. Traduire les deux attributs séparément produisait deux traits parallèles :

```
Eleve --> "cours 0..*" Cours        % ce que disait Eleve
Cours --> "inscrits 1..*" Eleve     % ce que disait Cours
```

La paire est désormais fusionnée, et le dessin porte le trait unique qu'UML décrit :

```
Eleve "inscrits 1..*" --> "cours 0..*" Cours
```

La fusion a lieu au **rendu**, et non à l'extraction : le modèle doit continuer de dire fidèlement ce que chaque classe déclare, puisque c'est lui que le processeur reconstruit pour vérifier une annotation.

Ce qui l'empêche. Une extrémité source ne sait porter qu'un rôle et une multiplicité : le losange d'une composition se retrouverait du mauvais côté, un libellé appartient au trait entier, une contrainte qualifie une extrémité que la syntaxe ne sait pas décorer. On ne fusionne donc que si le second lien **n'ajoute rien qu'un rôle et une multiplicité**, et si le premier n'a pas déjà écrit son propre côté source. Sinon les deux traits restent, et un avertissement le dit en nommant les deux attributs : perdre en silence ce que l'auteur a écrit serait le seul vrai défaut.

7 La Classe d'Association : `@UMLAssociationClass`

7.1 Principe

Identifie une classe comme porteuse des attributs d'une relation entre deux autres (table de jointure enrichie). En Java, il est impossible de lier un objet à un lien : il faut une classe intermédiaire, que cette annotation fait dessiner selon la norme UML — une ligne en pointillés reliant le lien principal à la classe.

7.2 Paramètres

`source` (Class) et `target` (Class), les deux extrémités de la relation. Les deux sont **obligatoires** : une classe d'association qui ne connaîtrait qu'une extrémité ne qualifierait rien.

Le trait est dessiné par elle. Déclarer la classe d'association suffit : le lien entre les deux extrémités est tracé du même coup. Écrire en plus une `@UMLAssociation` entre `source` et `target` dessinerait un second trait.

Ce qui est refusé. Les deux extrémités doivent porter `@UMLClass`, et la classe annotée aussi. Aucune extrémité ne peut être la classe d'association elle-même : le lien serait rattaché à la boîte qui le qualifie. En revanche `source` et `target` peuvent désigner *la même* classe : l'association réflexive existe en UML.

7.3 Exemple

```
@UMLAssociationClass(source = Etudiant.class, target = Cours.class)
@UMLClass
public class Inscription {
    private LocalDate dateInscription;
    private double noteObtenue;
}
```

8 La Multiplicité d'une propriété : `@UMLMultiplicity`

8.1 Principe

Déclare combien d'objets une propriété tient, entre crochets à côté de son nom : sur un attribut, le nombre d'objets tenus ; sur une méthode, le nombre d'objets rendus.

```
@UMLClass(strategy = UMLVisibility.PRIVATE)
public class CanonicalMnemonic {
    @UMLMultiplicity("0..*")
    private String deprecatedAliases;   // - deprecatedAliases: String
   [0..*]

    @UMLMultiplicity("0..1")
    private int ucp;                    // - ucp: int [0..1]

    @UMLMultiplicity("0..*")
    public String names() { ... }       // + names(): String [0..*]
}
```

8.2 Pourquoi c'est utile ?

La multiplicité d'un *lien* se déclare déjà par `targetCardinality`. Mais toutes les propriétés multiples ne sont pas des liens : un ensemble de chaînes ou un champ facultatif n'ont **aucune classe à l'autre bout**. Cette annotation dit le nombre, et rien que le nombre. Elle reste par ailleurs le seul moyen de qualifier une **méthode** : `names(): String [0..*]`.

8.3 Paramètres

`value` (String) : la multiplicité, dans le vocabulaire d'UML — `"1"`, `"0..1"`, `"*"`, `"1..*"`, `"2..5"`. C'est la même grammaire que celle des multiplicités de lien, vérifiée par la même règle. Le nom `value` permet la forme courte `@UMLMultiplicity("0..*")`.

8.4 Ce qu'elle ne déduit pas, et ce que le projet peut traduire

Une partie de la multiplicité **se lit** : `Optional<X>` veut dire « zéro ou un », `Set<X>` « zéro ou plusieurs ». La règle `multiplicity brackets` les transcrit pour tout le projet. Elle ne dit jamais *plus* que le type — `0..*` et non `1..*` — et l'annotation l'emporte sur elle. Au passage

du conteneur à l'élément, un type enveloppe redevient primitif : `Optional<Integer>` s'écrit `int [0..1]`, Java interdisant `Optional<int>`.

8.5 Combinaison avec les autres annotations

— **Avec `@UMLAssociation` : erreur de compilation**. `targetCardinality` donne déjà la multiplicité ; deux valeurs se contrediraient.

— **Avec `@UMLDerived` et `@UMLConstraint`** : sans difficulté — `- /etiquettes: String [1..*] {sans doublon}`.

— **Sur un membre non affiché : avertissement**.

9 Les Éléments Dérivés : `@UMLDerived`

9.1 Principe

Indique qu'un attribut, une méthode ou une association est dérivé — calculé, non stocké. UML le note d'un slash (`/`), que Java n'accepte pas dans un identifiant : l'annotation demande ce préfixe au rendu. Elle ne s'applique pas à une classe, la notion n'existant pas en UML. **Aucun paramètre.**

La barre oblique appartient au **nom** de la propriété, et à lui seul : ni le type, ni la visibilité, ni la multiplicité ne la portent. Sur un lien dessiné en trait, elle précède le rôle cible ; dans une boîte, le nom de la ligne.

9.2 Exemple

```
@UMLClass(strategy = UMLVisibility.PRIVATE)
public class Personne {
    private LocalDate dateNaissance;

    @UMLDerived
    private int age; // Affiche en UML sous la forme : - /age: int

    @UMLDerived
    @UMLAssociation(target = Commande.class, targetRole = "encours",
                    targetCardinality = "0..*")
    private List<Commande> commandesEnCours; // Trait portant "/encours
    0..*"

    @UMLDerived
    @UMLAssociation(target = Commande.class, targetCardinality = "0..*",
                    notation = UMLNotation.ATTRIBUTE)
    private List<Commande> commandesSoldees; // - /commandesSoldees:
    Commande [0..*]
}
```

10 Les Contraintes Métier : `@UMLConstraint`

10.1 Principe

Ajoute une règle formelle, écrite entre accolades `{ }`, sur une classe, un attribut ou un lien — certaines règles métier ne s'expriment que textuellement (`{subset}`, « solde positif »).

10.2 Paramètres

`rule` (String) : la règle, **écrite nue**. Les accolades sont ajoutées par le générateur ; les écrire soi-même les afficherait en double, ce dont le compilateur avertit. Annotation **répétable**. Si

l'élément porte aussi `@UMLAssociation`, la contrainte est rattachée au lien.

10.3 Où la contrainte est écrite

Trois positions, pour une seule notation :

— **sur un attribut** : à la suite de sa ligne — `- solde: double {toujours positif}` ;

— **sur un attribut porteur de `@UMLAssociation`** : là où l'association est dessinée — à l'extrémité cible du trait sous `ARROW`, à la fin de la ligne sous `ATTRIBUTE` —, après la multiplicité. Rien ne distingue un `{subset}` d'un `{ordered}` : c'est ce qui a permis de retirer le paramètre `isOrdered` ;

— **sur une classe** : dans une bulle accolée à la boîte, PlantUML n'ayant pas de notation propre.

Une contrainte posée sur un attribut non affiché ne serait écrite nulle part : le compilateur avertit.

10.4 Exemple

```
public class CompteBancaire {

    @UMLConstraint(rule = "solde >= decouverAutorise")
    private double solde; // Affichera {solde >= decouverAutorise} a cote
   de l'attribut
}
```

11 Les Liens Transversaux : `@UMLDependency`

11.1 Principe

Déclare une dépendance vers une classe qui n'est pas un attribut — utilisée localement dans une méthode. C'est la seule façon de révéler qu'une classe en utilise une autre sans en stocker l'état, le corps des méthodes ayant disparu à la compilation.

11.2 Paramètres

`target` (Class), la classe ciblée, et `label` (String), la raison du lien (ex. `"utilise"`). Annotation **répétable**.

11.3 Exemple

```
@UMLDependency(target = ServiceBancaire.class, label = "facture via")
public class Panier {
    public void valider() {
        ServiceBancaire sb = new ServiceBancaire();
        sb.payer();
    }
}
```

12 La Documentation Visuelle : `@UMLNote`

12.1 Principe

Génère une bulle de commentaire rattachée à une classe, un attribut ou une méthode. Un diagramme explique le « comment » (la structure), la note explique le « pourquoi ».

12.2 Paramètres

— `text` (String) : le contenu. Obligatoire et non vide — une note sans texte n'est qu'un carré jaune posé au hasard.

— `direction` (Enum) : `UMLDirection.NONE`, le défaut, place la bulle **à droite**, ce qui perturbe le moins la lecture verticale de l'héritage.

Annotation **répétable**.

12.3 Ce à quoi une note peut s'accrocher

Une bulle ne se dessine qu'attachée à un élément présent. Deux cas la laissent sans support, et le compilateur avertit : le **membre visé n'est pas affiché** ; ou l'**attribut visé porte une `@UMLAssociation` dessinée en trait** — il a quitté la boîte, et les notes rattachées à un lien sont hors périmètre. Poser la note sur la classe est alors la seule façon d'en parler. Sous `ATTRIBUTE`, l'association *est* une ligne de la boîte : la note s'y accroche, en visant le **nom affiché**.

12.4 Exemple

```
public class MoteurDeRecherche {
    @UMLNote(text = "Algorithme critique, ne pas modifier", direction =
    UMLDirection.RIGHT)
    public void indexer() {
    }
}
```

13 Le Regroupement Logique : `@UMLPackage`

13.1 Principe

Regroupe visuellement des classes dans un rectangle nommé. Sur un projet de quarante classes, regrouper par domaine métier divise la complexité visuelle sans forcer à restructurer les dossiers Java.

13.2 Paramètres

— `name` (String) : nom du groupe, **obligatoire**. C'est lui qui fait l'identité du groupe : deux classes déclarant le même `name` entrent dans le même rectangle.

— `color` (String) : couleur de fond, écrite comme celle d'une boîte et vérifiée par la même règle. Deux couleurs pour un groupe : la première l'emporte, avertissement.

Le groupe n'est pas le paquetage Java. Il est déclaré, jamais déduit : le découpage qui fait un bon diagramme n'est pas toujours celui qui fait une bonne arborescence de fichiers. Deux classes de paquetages différents peuvent figurer dans le même rectangle, et inversement.

Ce que le groupe ne change pas. Les boîtes sont écrites à l'identique et les traits continuent de le traverser : un rectangle range les boîtes, il ne les modifie pas.

13.3 Exemple

```
@UMLPackage(name = "Module Paiement", color = "#EEEEEE")
@UMLClass
public class Facture {
}
```

14 Les règles du projet : le fichier `umlg.rules`

14.1 Principe

Certaines décisions ne portent pas sur un élément mais sur le projet entier : « dans ce diagramme, un `ImmutableList` s'affiche `List` ». Elles se déclarent dans un fichier facultatif, `umlg.rules`, posé **à la racine du répertoire analysé** — pour un projet Maven ordinaire, `src/main/resources/umlg.rules`, que la construction recopie dans `target/classes`.

Une annotation est le bon outil pour ce qui se décide au cas par cas, et le mauvais pour ce qui vaut partout : une règle générale s'écrit une fois, à un endroit.

14.2 Le partage avec les annotations

La bibliothèque porte sur les annotations ; le fichier est le **cas rare**. Le critère est unique :

Cette décision désigne-t-elle un élément par son nom ?

Si oui, c'est une annotation, sans exception. Si non — la décision vaut pour tout le projet, ou pour une vue —, c'est une règle. Trois conséquences : **aucune règle ne désigne un membre nommément** (`hide Representation.string` n'existe pas) ; **aucune règle ne range une classe nommée** (il n'y a pas de `group package`) ; **aucune règle ne fabrique une boîte ni un lien**. D'où une priorité en une phrase, sans exception :

L'annotation l'emporte toujours sur la règle.

`@UMLInclude` ramène tout ce qu'un `hide` écarte ; `@UMLMultiplicity` l'emporte sur `multiplicity brackets` ; `@UMLClass(strategy)` sur `strategy`. Le récapitulatif de ce que les règles ne touchent pas tient en quatre lignes :

Ce qu'on veut	Où cela s'écrit
Renommer une boîte	`@UMLClass(name = ...)`
Renommer un membre	`@UMLInclude(name = ...)`
Taire un membre précis	`@UMLExclude`
Ranger des classes dans un rectangle	`@UMLPackage(name = ...)`

14.3 Forme du fichier

Une règle par ligne. Les lignes vides sont ignorées, ainsi que les commentaires, ouverts par `#`. Une règle commence par un mot-clé qui dit sa nature. Le vocabulaire est **fermé** : un mot inconnu arrête la génération plutôt que d'être ignoré. Le fichier doit être encodé en **UTF-8**.

14.4 Le vocabulaire complet

Onze mots-clés, et **aucun** qui refasse ce qu'une annotation sait faire. Le tableau donne la **forme** de chaque règle, et non un exemple : ce qui est écrit en `fonte à chasse fixe` s'écrit tel quel, ce qui est en *<italique entre chevrons>* est à remplacer par l'auteur, et une barre verticale sépare des valeurs au choix. Les exemples viennent ensuite (section 14.5).

Forme de la règle	Ce qu'elle fait
`hide` *<motif>*`,` *<motif>*`...`	Écarte les **membres** — méthodes comme attributs — dont le nom suit l'un des motifs, où `*` remplace n'importe quelle suite de caractères. Ex. : `hide get*, log, *Cache`

Forme de la règle	Ce qu'elle fait
`omit visibility`	Tait les symboles `+`, `-`, `#`, `~`.
`omit parameter names`	Tait les noms des paramètres : `from(KeyboardMap)`.
`omit return void`	Efface le type rendu : `valider()`.
`strategy` *<none / public / protected / package / private>*	La stratégie des classes qui ne déclarent pas la leur. Ex. : `strategy public`
`type` *<type du code>* `->` *<nom affiché>*	Renomme *n'importe quel* type dans les boîtes, chevrons compris — les deux noms sont libres. Ex. : `type ImmutableList -> List`, `type Optional -> Opt`
`order` *<declaration / alphabetical>*	L'ordre des membres dans les boîtes ; `alphabetical` est le défaut.
`multiplicity brackets`	`Set<X>` s'écrit `X [0..*]`, `Optional<X>` `X [0..1]`.
`title` *<texte libre>*	Titre de la vue intégrale, pris tel quel jusqu'en fin de ligne.
`direction` *<top-to-bottom / left-to-right>*	Son sens de lecture.
`show empty members`	Rétablit les compartiments vides.
`view "`*<nom>*`" =` *<sélecteur>*`,` *<sélecteur>*`...`	Déclare une vue et ce qu'elle retient, dans l'ordre.
`view "`*<nom>*`" =` *<sélecteurs>* `-` *<sélecteurs>*	La même, moins ce qui suit le tiret (entouré d'espaces).
`view "`*<nom>*`" direction` *<sens>*	Le sens de lecture de cette vue.
`views by package`	Une vue par paquetage, sans rien déclarer.

Un *<sélecteur>* s'écrit de trois façons, et de trois seulement : *<Classe>* par son nom simple ou complet, `package` *<paquetage>*, ou `around` *<Classe>* (section 16). Un *<motif>* et un *<nom de type>*, eux, ne sont contraints par aucune liste : ce sont les noms du projet analysé.

14.5 Chaque mot-clé par l'exemple

`strategy` : la visibilité de tout le projet. Elle donne la stratégie que prennent les classes dont le `strategy` vaut `PROJECT`, c'est-à-dire le défaut de `@UMLClass`. Sans elle, obtenir un diagramme peuplé demanderait d'écrire `@UMLClass(strategy = ...)` sur chaque classe. Une classe qui déclare la sienne l'emporte, dans les deux sens.

```
strategy public     # aussi : none, protected, package, private
```

Sans aucune ligne `strategy`, le défaut ne dessine aucun membre et les boîtes sortent vides — la surprise type d'un premier essai, que la génération **signale**. Un `strategy none` **écrit** produit le même dessin sans le signalement : une ligne écrite est un choix, pas un oubli.

`hide` : ce que le projet tient pour du bruit. Rien n'est masqué sans demande (section 3.4). Un `hide` fait toujours la même chose — le membre **disparaît de la boîte** — et ne s'écrit que d'une seule façon : un ou plusieurs **motifs de noms**, séparés par des virgules. L'étoile y remplace n'importe quelle suite de caractères, y compris aucune ; tout le reste est pris au pied de la lettre, et la casse compte.

```
# Forme : hide <motif>, <motif>... Les motifs sont les noms du projet.
hide get*, set*, is*        # les accesseurs, par leur convention de nom
hide toString, hashCode     # les methodes d'Object qu'on ne lit pas
hide with*, visit*, *Cache  # les conventions propres au projet
```

La bibliothèque ne connaît donc aucune famille de membres ; elle ne connaît que les noms du

projet analysé, et c'est l'auteur du fichier qui décide ce qu'ils désignent. Un seul mécanisme à comprendre, et il se relit sans documentation : la ligne dit exactement ce qu'elle masque.

Le prix de cette simplicité est à connaître. Un motif ne regarde que le **nom**, jamais la signature : `hide get*` emporte aussi un `getPrix(Client)` qui est une méthode métier, et `hide equals` emporte la surcharge `equals(Client)` avec l'`equals(Object)` d'origine. Le rattrapage est le même que partout ailleurs, et il tient en une annotation : `@UMLInclude` sur le membre à conserver l'emporte sur la règle. Un motif vise **les méthodes et les attributs indifféremment** : il n'y a pas de `hide method` ni de `hide field`, et le rare membre pris à tort se ramène par `@UMLInclude`. L'étoile s'écrit : `hide get` ne masque que le membre nommé exactement `get`, `getName` demande `hide get*`. L'oubli n'est pas silencieux — une ligne qui n'a rien masqué de tout le scan est signalée en fin de génération. Enfin, aucun motif ne désigne un membre précis : `hide Representation.string` est refusé, `@UMLExclude` est fait pour cela.

`omit` : ce qui n'est pas écrit sur la ligne. Trois réglages ne font disparaître aucun membre : ils retirent **un morceau de la ligne** d'un membre qui reste affiché.

```
omit visibility         # "valider()" et non "+ valider()"
omit parameter names    # "from(KeyboardMap)" et non "from(m : KeyboardMap)"
omit return void        # "valider()" et non "valider() : void"
```

Le verbe est distinct de `hide` à dessein : il dit à lui seul qu'aucun `@UMLInclude` ne les contredira, puisqu'il n'y a rien à ramener. `omit return void` ne vaut que pour le **retour**, et que pour `void` : UML tient ce type pour facultatif. Effacer un type de retour utile — `names(): String` devenant `names()` — ferait perdre au dessin ce que la boîte existe pour montrer.

`type` : les types affichés. Remplace un nom de type par un autre, **partout où ce type paraît dans une boîte** : attributs, types de retour, paramètres, y compris à l'intérieur des chevrons.

```
# Forme : type <type du code> -> <nom affiche>. Les deux noms sont libres.
type ImmutableList -> List
type com.google.common.collect.ImmutableSet -> Set
# "ImmutableList<ImmutableSet<String>>" s'ecrit "List<Set<String>>"
```

Le nom de gauche s'écrit au choix simple ou complet ; le nom complet l'emporte sur le nom simple, ce qui permet de viser une classe précise quand deux paquetages en ont une homonyme. La règle ne touche pas le **nom d'une boîte**, qui relève de `@UMLClass(name)` : elle ferait sinon diverger le type écrit dans une boîte du nom de la boîte d'à côté.

```
order declaration    # l'ordre du source, relu dans le fichier .class
# order alphabetical # le defaut ; s'ecrit aussi, pour qui veut le dire
```

```
multiplicity brackets
# Set<String> aliases    ->  aliases : String [0..*]
# Optional<Integer> ucp  ->  ucp : int [0..1]
```

Elle ne dit jamais plus que le type — `0..*` et non `1..*` — et `@UMLMultiplicity` l'emporte sur elle (section 8).

`title`, `direction`, `show` : le dessin d'ensemble. Ces trois réglages étaient autrefois portés par une annotation `@UMLDiagram` posée sur un `package-info.java`. Ils ne disent rien d'une classe : ils valent pour le dessin entier.

```
title Architecture du module Facturation
direction left-to-right   # aussi : top-to-bottom, le defaut
show empty members        # les compartiments vides, masques par defaut
```

Le titre est pris tel quel, espaces compris : c'est une phrase, pas un identifiant. Le masquage des compartiments vides n'est pas neutre : sans stratégie déclarée, une boîte n'a aucun membre, et ne pas masquer dessinerait deux rectangles vides sous chaque nom.

`view` et `views` : les vues. Détaillées en section 16.

```
# Forme : view "<nom>" = <selecteur>, <selecteur>... [- <selecteurs>]
view "Le domaine" = package com.banque.modele
view "Facturation" = Facture, LigneFacture, Client
view "Autour de Compte" = around Compte
view "Le domaine allege" = package com.banque - XkbSymbolsReader
view "Facturation" direction left-to-right
views by package
```

14.6 Le fichier minimal

Le tableau se lit d'un bout à l'autre, mais personne n'a besoin de tout. Trois lignes suffisent au cas nominal :

```
strategy public
hide get*, set*
type ImmutableList -> List
```

La première peuple les boîtes, la deuxième les allège, la troisième dit que l'implémentation ne regarde pas le lecteur. Tout le reste est de l'ajustement, à écrire le jour où le dessin le réclame.

14.7 Un fichier complet

```
# ---- Le dessin d'ensemble ----
title Architecture du module Facturation
direction left-to-right

# ---- Ce que les boites montrent ----
strategy public
hide get*, set*, is*
hide toString, hashCode, equals
hide with*, visit*, log, LOGGER
omit return void
omit visibility
omit parameter names
order declaration
multiplicity brackets
type ImmutableList -> List

# ---- Les vues ----
view "Le domaine" = package com.exemple.modele
view "Facturation" = Facture, LigneFacture, Client
view "Facturation" direction left-to-right
view "Autour de Facture" = around Facture

views by package
```

14.8 Ce qui est refusé

Le fichier est écrit à la main, sans compilateur pour le relire : chaque refus nomme le fichier, le numéro de ligne et la forme attendue. Un fichier fautif **arrête la génération** plutôt que d'être appliqué à moitié.

Situation	Raison
Fichier qui n'est pas en UTF-8	Le premier accent d'un commentaire en CP-1252 rendait la lecture illisible derrière une exception générique.
Mot-clé inconnu (`couleur Client -> rouge`)	Le vocabulaire est fermé ; le message rappelle les mots-clés connus.
Stratégie inconnue, ou `strategy project`	`project` est ce qu'une classe écrit pour renvoyer à cette ligne, qui ne peut pas se renvoyer à elle-même.
Deux lignes `strategy`, deux `title`, deux `order`, deux fois le même `omit`, deux règles sur le même type	La seconde l'emporterait en silence, ou ne demanderait rien de plus ; un fichier qui se répète est un fichier que personne ne relit.
`title` sans texte	Un titre vide ne s'écrit pas.
`hide` sans aucun motif	La ligne ne dit pas ce qu'elle masque ; le message rappelle la forme attendue.
Motif vide, ou réduit à `*`	Une virgule orpheline, le plus souvent ; `*` masquerait toute la boîte.
Motif contenant un point (`hide Representation.string`)	Une règle ne désigne pas un membre nommément ; le message renvoie à `@UMLExclude`.
`omit`, `order`, `direction`, `show` ou `multiplicity` de valeur inconnue	Vocabulaire fermé ; le message rappelle les formes admises.
`view` sans nom entre guillemets, ou sans `=`	La vue ne dit pas comment elle s'appelle, ou ce qu'elle contient.
Vue déclarée deux fois, ou `direction` avant son contenu	Le sens de lecture d'une vue inexistante ne s'applique à rien.
Sélecteur ne désignant aucune classe	Une classe renommée laisserait un nom mort, et le diagramme se dessinerait amputé sans que rien ne le signale.
Flèche `->` absente, l'une des moitiés vide, ou espace au milieu d'un nom de type	La règle ne dit pas ce qu'elle remplace, ou par quoi ; l'espace trahit une flèche oubliée.

Un cas échappe à ce tableau parce qu'il ne peut pas être un refus : une règle de nommage dont le nom ne correspond à rien — `type ImutableList -> List`, `hide LOGGR`, `hide accesseurs`, l'étoile oubliée d'un `hide get` — est syntaxiquement irréprochable, et un motif a le droit de ne viser personne aujourd'hui. Elle n'est donc pas refusée, mais la génération relève en fin de scan les lignes restées sans effet et les signale (section 16.9).

14.9 Ce qui décide d'une règle future

Une règle candidate doit passer trois questions : *désigne-t-elle un élément par son nom* — alors c'est une annotation ; *dit-elle quelque chose du modèle* — sinon c'est une option d'appel ; *une règle existante la fait-elle déjà* — alors elle ne s'ajoute pas. Le vocabulaire s'est ainsi raccourci plus souvent qu'il ne s'est allongé : trois mots-clés ont été retirés (`group`, `output`, `declare`), et `hide method`/`hide field` fondus en un seul motif.

15 L'apparence du diagramme : `UmlStyle`

15.1 Principe

L'habillage — couleurs des boîtes, épaisseur et teinte des traits, police — ne se déclare ni sur un élément ni dans le fichier de règles, mais **à l'appel du générateur**. C'est le seul des

trois endroits qui ne touche pas au modèle : les mêmes boîtes, les mêmes traits, les mêmes membres, vêtus autrement.

```
UMLGenerator.builder()
    .sourceDirectory("target/classes")
    .outputFile("architecture.svg")
    .style(UmlStyle.UMLG)   // facultatif ; PLANTUML par defaut
    .build()
    .generate();
```

Deux diagrammes engendrés dans les deux styles portent la même information : l'apparence relève de qui *regarde* le diagramme, et peut changer d'une génération à l'autre sans qu'une ligne de code bouge. Le vocabulaire est **fermé** — deux valeurs, aucun accès aux `skinparam` (section 18).

Valeur	Ce qui est dessiné	Quand la choisir
`PLANTUML` (défaut)	Ce que PlantUML dessine quand on ne lui dit rien : boîtes grises, pastille `C` ou `I`, traits par défaut.	Le cas nominal : un utilisateur qui n'a rien demandé retrouve le rendu qu'il reconnaît.
`UMLG`	Bandeau bleu pâle sur fond blanc cassé, bords bleu foncé, coins arrondis, pas d'ombre, nom des liens en italique coloré, pastille masquée.	À l'impression, ou sur un fond clair : plus sobre, au prix d'un repère visuel en moins.

15.2 Ce qui échappe au choix

Deux réglages sont posés dans les deux cas, parce qu'ils touchent au sens et non à l'apparence : `classAttributeIconSize 0`, qui écrit les visibilités `+`, `-`, `#`, `~` plutôt qu'en pastilles colorées propres à PlantUML ; et l'**espacement**, calculé sur la plus longue étiquette du diagramme, sans quoi les étiquettes de lien se chevaucheraient. Le masquage des compartiments vides leur ressemble sans leur appartenir : les deux apparences le suivent, elles ne le décident pas — c'est `show empty members` qui le lève.

16 Les vues : plusieurs diagrammes pour un projet

16.1 Principe

Un projet de trente classes donne un dessin que personne ne lit ; ce qu'on lit, ce sont quelques diagrammes de cinq à dix boîtes, chacun répondant à une question. La bibliothèque produit donc, en une exécution, **un dossier de diagrammes** plutôt qu'une image :

```
diagrammes/
|-- projet.svg          <- la vue integrale, toujours produite
`-- vues/
    |-- le-domaine.svg
    |-- autour-de-compte.svg
    `-- com.banque.modele.svg
```

La **vue intégrale** est toujours écrite, à la racine : c'est l'inventaire du projet. Elle porte le titre et le sens de lecture déclarés par `title` et `direction`. Les autres vues vivent dans un sous-dossier ; chacune porte son propre titre — son nom — et son propre sens de lecture.

16.2 Déclarer une vue

```
# Ce que la vue retient, dans l'ordre ou on veut le lire.
view "Le domaine" = package com.banque.modele
view "Facturation" = Facture, LigneFacture, Client
view "Autour de Compte" = around Compte

# Ce qu'elle retire, apres un tiret entoure d'espaces.
view "Le domaine sans les lecteurs" = package com.banque - XkbSymbolsReader

# Son sens de lecture, sur une ligne a part.
view "Facturation" direction left-to-right
```

Trois façons de désigner des classes, et trois seulement :

Sélecteur	Ce qu'il retient
`Facture`	Une classe, par son nom simple ou complet. La forme qui reproduit un diagramme tenu à la main : on choisit, une par une.
`package com.exemple`	Toutes les classes du paquetage, correspondance exacte — un sous-paquetage n'entre pas.
`around Facture`	La classe et ses voisines immédiates. Le rayon est de un, délibérément : à deux, le voisinage d'une classe centrale redevient le projet entier.

Un voisinage peut être vide. Les voisins d'`around` sont les extrémités des **relations déclarées** — `@UMLAssociation`, `@UMLDependency`, héritages entre classes annotées — et non les types que le code manipule. Une classe très entourée dans le code peut donc n'avoir aucun voisin déclaré, et la vue se réduit à une boîte seule. Le diagramme est exact, mais la cause ne se lit pas sur le dessin : la génération **avertit** et nomme les deux remèdes.

Un sélecteur qui ne désigne aucune classe **arrête la génération** : une classe renommée y laisserait un nom mort, et le diagramme se dessinerait amputé sans que personne ne le sache.

16.3 L'ordre d'écriture est l'ordre de lecture

Les classes d'une vue sont dessinées dans l'ordre où elles sont écrites. C'est le seul moyen d'exprimer quelque chose que le code ne contient pas : le moteur de rendu place déjà correctement ce qui est relié, mais il ne peut pas deviner l'ordre de deux boîtes que *rien* ne relie — or cet ordre-là est celui de l'exposé qui accompagne le dessin.

La bibliothèque enchaîne donc par un lien invisible les **blocs** de la vue, dans l'ordre où leur première boîte est écrite. Un bloc est un groupe de boîtes que les traits relient, directement ou de proche en proche.

Pourquoi des blocs, et non des boîtes. Trois classes filles d'un même parent ne sont pas reliées *entre elles* : enchaînées deux à deux, elles se retrouveraient empilées, alors que le moteur les dispose en éventail sous leur parent. Ainsi une vue retenant `KeysymEntry` et ses trois filles, puis `KeyboardMap`, puis `XkbSymbolsReader`, reçoit exactement deux liens invisibles, et pas un de plus. C'est ce qui a permis de retirer `@UMLLayout`, seule annotation à ne rien dire du modèle : l'enchaînement des blocs ordonne ce que le moteur ne peut pas deviner, sans rien demander à l'auteur, et se recalcule seul quand le code change.

L'ordre quand rien n'est écrit. Un sélecteur `package` ou une vue calculée ne portent aucun ordre : leurs boîtes sont rangées par nom complet. Exigence de reproductibilité, comme pour les membres d'une boîte : le parcours du système de fichiers ne promet aucun ordre.

16.4 Les vues calculées

```
views by package
```

Une vue par paquetage Java, sans rien déclarer : c'est le découpage que suivent la plupart des diagrammes tenus à la main. Il ne dispense pas de déclarer des vues, celles-ci retenant *tout* le paquetage là où un auteur en écarterait les classes techniques.

16.5 Les liens qui sortent d'une vue

Une relation dont l'autre bout n'est pas dans la vue **disparaît, sans un mot** : restreindre la vue *est* la demande de l'auteur. Le contraire — dessiner la boîte manquante pour sauver le trait — ferait déborder chaque vue de proche en proche, jusqu'à ce qu'elles se ressemblent toutes.

16.6 Passer d'une vue à l'autre

Chaque boîte porte un **lien cliquable** vers la vue qui la détaille : le dossier devient une documentation qui se parcourt. La cible est la première vue déclarée qui retient cette boîte ; une vue ne renvoie jamais vers elle-même. Les chemins sont relatifs, si bien que le dossier reste déplaçable.

16.7 Quand le dessin devient illisible

Une vue dont les boîtes s'alignent sur une seule rangée donne un ruban de plusieurs milliers de pixels de large sur quelques centaines de haut : rien n'a échoué, la source est correcte, et l'auteur ne le découvre qu'en ouvrant le fichier. Deux garde-fous.

L'enchaînement s'arrête à trois blocs. Ordonner deux ou trois groupes dit une intention ; en ordonner cinq construit une file indienne que le moteur déroule sur toute la largeur. Au-delà de trois, la vue n'est donc plus contrainte du tout : sur un cas réel, cinq blocs donnaient 3570 pixels de large pour 289 de haut, les mêmes boîtes sans contrainte 1578 sur 824.

Les proportions sont mesurées. La bibliothèque relit les dimensions de ce qu'elle vient d'écrire et **avertit** lorsque le rapport dépasse quatre pour un, sur une image assez grande pour que cela compte. Elle ne corrige rien : les causes sont plusieurs — un sens de lecture inadapté, des boîtes trop larges, une vue trop peuplée — et le choix appartient à l'auteur. La mesure vaut pour les deux modes de sortie.

16.8 Ce que la génération dit d'elle-même

Un compte rendu, en deux lignes :

```
12 diagram(s) written to /chemin/diagrammes
  21 class(es) drawn, 23 ignored (no @UMLClass)
```

Le second chiffre est celui que personne n'avait : une classe volontairement écartée et une classe qu'on a oublié d'annoter se ressemblent trait pour trait, c'est-à-dire qu'elles sont absentes toutes les deux. Sous une douzaine, elles sont nommées ; au-delà, la génération écrit `ignored-classes.txt` à la racine de la sortie, avec **toutes** les classes ignorées, une par ligne, et un fichier périmé d'une génération précédente est retiré.

Le compte rendu est rendu, non imprimé. `generate()` rend le `RapportDeGeneration` et n'écrit nulle part : choisir un canal n'appartient pas à une bibliothèque. La ligne de commande l'écrit sur la sortie standard, le plugin Maven le passe à son log. Les **avertissements** suivent le même principe.

La source PlantUML est conservée. Chaque image est accompagnée de sa source, sous le même nom et l'extension `.puml` : sans elle, pas de retouche possible pour un essai, et le diff entre deux générations se faisait sur du SVG illisible. Les `.puml` suivent le sort de leurs images.

16.9 Ce que la génération signale

Les **avertissements** émis pendant le scan partagent une définition : le diagramme est *exact*, mais il dit moins que ce que son auteur croit avoir demandé, et rien sur le dessin n'en montrerait la cause. Et une même retenue : chacun se tait dès que le silence est juste — un choix écrit n'est jamais signalé.

Situation	Ce que l'auteur découvrirait sinon sur l'image
Des boîtes sans membre, aucune `strategy` déclarée nulle part	Le défaut ne dessine aucun membre. Le message nomme les boîtes et les trois remèdes. Il se tait devant un `strategy none` **écrit**, et devant une classe sans aucun membre.
Aucun paramètre d'aucune méthode ne porte de nom	Projet compilé sans `-parameters` (section 2.2). Il se tait si un seul nom existe, ou si `omit parameter names` l'a demandé.
Des `.class` que le chargeur n'a pas su lier	Une dépendance absente du classpath, ou une compilation partielle. Ces classes sont écartées et nommées plutôt que de faire échouer le scan entier. L'avertissement dit aussi ce qu'on *ne peut pas* savoir : rien ne dit si elles portaient `@UMLClass`.
Deux boîtes portant le même nom affiché	Deux classes homonymes. Les alias restent distincts — chaque trait aboutit à la bonne boîte — mais le lecteur voit deux rectangles identiques. `@UMLClass(name)` est là pour que l'auteur tranche.
Une vue `around` sans voisin	Les voisins viennent des relations déclarées, pas des types du code.
Une règle de nommage restée lettre morte	Un `type` ou un motif de `hide` qui n'a rien touché pendant le scan : une faute de frappe ou l'étoile oubliée d'un `hide get`. L'avertissement cite chaque ligne telle qu'elle a été écrite.
Une vue aux proportions illisibles	Le ruban décrit ci-dessus.

Ce qui l'arrête, en revanche. Une erreur dit qu'aucun diagramme ne peut être produit. Arrêtent donc la génération : une annotation contradictoire, une couleur inconnue de PlantUML, un fichier de règles fautif ou qui n'est pas en UTF-8, une vue nommant une classe absente, un répertoire où aucune classe ne porte `@UMLClass` — le message nomme alors les classes lues.

S'y ajoute un cas qui n'est pas une faute de l'auteur : **Graphviz introuvable**. Quand il manque, PlantUML ne lève rien — il dessine une image portant « Cannot find Graphviz », que ni sa taille ni son format ne distinguent d'un dessin réussi. La génération vérifie donc que le moteur est en état avant de dessiner, une fois par exécution, et dit quoi installer : une image d'erreur livrée comme un diagramme est le pire des résultats, puisque rien ne la dénonce.

16.10 Ce que devient la génération précédente

```
UMLGenerator.builder()
    .outputDirectory("diagrammes")
    .keepPrevious(true)     // facultatif ; false par defaut
```

Par défaut, les diagrammes précédents sont effacés : le dossier décrit toujours l'état courant du code, et une vue retirée du fichier de règles ne laisse pas derrière elle une image que plus rien ne met à jour — laquelle ment d'autant mieux qu'elle a l'air d'être à sa place. Seuls les fichiers que la bibliothèque écrit sont retirés : un `LISEZ-MOI` posé dans le dossier y reste. Sous

`keepPrevious`, la génération est écrite dans un dossier voisin numéroté — `diagrammes-2`, `diagrammes-3` —, ce qui permet de comparer avant et après une modification. Depuis Maven, `-Dumlg.keepPrevious=true`. Ce réglage n'est pas une règle du projet : il ne dit rien du *modèle*, il dit ce que la *commande* fait des fichiers déjà présents.

16.11 Lancer la génération

Depuis Maven , ce qui est la forme à préférer : un **plugin dédié**, `umlg-maven-plugin`, dont les défauts disent l'usage — `target/classes` analysé, `diagrammes/` écrit à la racine du projet.

```
<plugin>
    <groupId>io.github.oliviercailloux</groupId>
    <artifactId>umlg-maven-plugin</artifactId>
    <version>1.0-SNAPSHOT</version>
    <executions>
        <execution>
            <goals><goal>generate</goal></goals>
        </execution>
    </executions>
</plugin>
```

puis `mvn compile umlg:generate`. Chaque réglage se surcharge par propriété : `-Dumlg.outputFile=arch` (un seul fichier), `-Dumlg.format=png`, `-Dumlg.outputDirectory=...`, `-Dumlg.skip=true`. Les chemins relatifs s'entendent depuis la racine du module et non du répertoire d'où Maven a été lancé : dans un réacteur, tous les modules écriraient sinon le même fichier. Le goal `umlg:help` énumère ces réglages.

Trois exigences pèsent sur le `pom` du projet analysé, et l'une est un piège. Il faut `maven.compiler.parameters` pour que les signatures portent le nom de leurs paramètres — mais il faut aussi **épingler la version** de `maven-compiler-plugin` : sans elle, Maven résout la 3.1, publiée en 2013 et antérieure au paramètre `parameters`, qui est alors ignoré en silence. Il faut enfin `showWarnings`, sans quoi Maven passe `-nowarn` à `javac` et fait taire tout ce que le processeur d'annotations a à dire, ses erreurs mises à part.

Le plugin transmet de surcroît à la bibliothèque le **classpath d'exécution** du projet : la réflexion ne s'arrête pas au répertoire des classes — charger une classe résout son parent, lire un attribut résout son type. Enfin, les paquetages exportés rendant des types Guava dans leurs signatures (`RapportDeGeneration.classesIgnorees()`, `UMLGenerator.couleursConnues()`), le `module-info` les rend lisibles par ses clients (`requires transitive`).

```
UMLGenerator <repertoire-analyse> [fichier.svg|dossier] [png|svg]
```

Le second argument dit à lui seul ce qu'on veut : un nom finissant par `.svg` ou `.png` demande ce seul fichier, tout autre nom demande un dossier de vues. Personne n'appelle « diagrammes » un fichier, ni « architecture.svg » un répertoire. Une faute de l'utilisateur y ressort comme un message et non comme une trace d'exécution.

Par programme , pour choisir l'apparence ou demander un dossier :

```
UMLGenerator.builder()
    .sourceDirectory("target/classes")
    .outputDirectory("diagrammes")
    .outputFormat(".svg")   // facultatif, .svg par defaut
    .build()
    .generate();
```

La forme d'origine, `outputFile("architecture.svg")`, reste disponible et n'écrit que le diagramme d'ensemble. C'est `outputDirectory` qui écrit toutes les vues.

17 La vérification à la compilation

17.1 Principe

La bibliothèque embarque un **processeur d'annotations** (JSR 269), c'est-à-dire du code qui s'exécute à l'intérieur de `javac`. Il n'engendre aucun fichier : il lit les annotations et parle, en pointant la valeur en cause dans le fichier source. Une annotation mal écrite est une **faute de code**, pas un incident d'exécution : la signaler à la compilation la met sous les yeux de son auteur, plutôt que quelques minutes plus tard au fond d'une pile d'exceptions — ou, pire, jamais, sur un diagramme silencieusement faux. Le processeur ne réécrit aucune règle : il construit les objets du modèle avec les valeurs lues et traduit en erreur de compilation l'exception que le modèle lève déjà. Les deux niveaux de vérification ne peuvent donc pas diverger, et le message affiché par `javac` est mot pour mot celui que la génération aurait produit.

17.2 Deux gravités

— **Erreur** : *ce code ne décrit aucun diagramme.* La compilation s'arrête.

— **Avertissement** : *ce code décrit un diagramme, mais vraisemblablement pas celui que son auteur croit.* La compilation continue — l'annotation fait toujours foi. Mais taire l'écart laisserait l'utilisateur le découvrir sur l'image, sans indice sur sa cause.

17.3 Ce qui arrête la compilation

Situation	Raison
Cible d'un lien dépourvue de `@UMLClass`	La cible a été écrite exprès ; son absence est un oubli.
Cible primitive (`int.class`) ou qui n'est pas une classe	Seule une classe du diagramme peut être une extrémité.
Multiplicité illisible (`"beaucoup"`) ou inversée (`"3..1"`)	Le vocabulaire des multiplicités est fermé.
Couleur inconnue de PlantUML	Vérifiée contre le catalogue réel du moteur.
Texte fait uniquement d'espaces (nom, rôle, label, stéréotype, titre)	Produirait une étiquette invisible.
`@UMLInclude(name)` et `targetRole` sur la même ligne	Deux noms pour la même ligne se contrediraient.
`@UMLPackage(name)` vide ou fait d'espaces	Un groupe est identifié par son nom.
`@UMLAssociationClass` dont une extrémité est la classe annotée	Le lien serait rattaché à la boîte qui le qualifie.
`@UMLInclude` et `@UMLExclude` sur le même membre	Intention contradictoire.
`asAttribute = true` sur une méthode à paramètres, sur une méthode `void`, ou sur un attribut	Les parenthèses retirées, il ne reste que le nom et le type, et il faut que les deux existent — un attribut en est déjà un.
`@UMLMultiplicity` et `@UMLAssociation` sur le même attribut	Deux multiplicités pour une seule propriété.
Annotation posée sur une classe dépourvue de `@UMLClass`	Le lien serait tracé depuis une boîte inexistante.

Situation	Raison
Dépendance d'une classe vers elle-même, ou deux fois la même	N'apprendrait rien, ou dessinerait deux traits superposés.

17.4 Ce qui produit un avertissement

Situation	Ce que l'auteur attendait, et n'aura pas
`@UMLInclude` sur un membre que la `strategy` affiche déjà, ou `@UMLExclude` sur un membre qu'elle masque déjà	La dérogation ne déroge à rien — sauf si elle **renomme** : renommer est un effet, et rien n'est alors signalé.
`@UMLInclude` sur un attribut porteur de `@UMLAssociation`	Sous `ARROW`, l'attribut est un lien et rien ne l'y ramènera ; sous `ATTRIBUTE`, il est déjà affiché. Un `name` qui renomme reste un effet réel.
`@UMLInclude(name)` reprenant le nom du code	Le renommage n'a pas eu lieu, et l'auteur le chercherait sur l'image.
`@UMLExclude` sur un attribut porteur de `@UMLAssociation`	Ce qui disparaît est le **lien** sous `ARROW`, la **ligne** sous `ATTRIBUTE`.
Paramètre de trait écrit sous `notation = ATTRIBUTE` (`type`, `label`, `direction`, `sourceRole`, `sourceCardinality`)	Une ligne de boîte ne porte qu'un nom, un type et une multiplicité : ces valeurs ne seront dessinées nulle part. Le message les nomme une à une.
`stereotype` contenant déjà `« »`, ou `rule` portant déjà ses accolades	Elles seraient dessinées en double.
Deux classes affichant la même étiquette	Deux rectangles que le lecteur ne peut pas distinguer.
`@UMLDependency` vers une classe déjà atteinte par une association **dessinée en trait**	Une association dit strictement plus qu'une dépendance. Sous `ATTRIBUTE`, rien n'est signalé.
Attribut de type collection sans `targetCardinality`	Ni le lien ni la ligne ne porteront de multiplicité ; seul le métier sait si c'est `0..*` ou `1..*`.
Multiplicité que le type contredit (une `List` pour `"0..1"`)	L'un des deux se trompe, et ce n'est pas au générateur d'en décider.
`target` absent du type de l'attribut	Souvent un copier-coller. Le lien sera tout de même dessiné vers la classe déclarée.
`@UMLDerived`, `@UMLMultiplicity`, `@UMLNote` ou `@UMLConstraint` sur un membre que le diagramme n'affiche pas	Une barre oblique, des crochets, une bulle ou des accolades ne se voient que si l'élément qu'ils marquent est dessiné.
`@UMLMultiplicity("")`	C'est ce que porte un membre qui n'en a pas : l'annotation ne demande rien.
`@UMLDerived` sur un lien **en trait** sans `targetRole`	La barre se pose devant le rôle cible ; sans rôle, rien ne peut la porter.
`@UMLNote` sur un attribut porteur d'une association **en trait**	L'attribut est devenu un trait, et une note ne s'accroche pas à un trait.
Deux classes du même groupe déclarant deux couleurs	Un rectangle n'en a qu'une : la première l'emporte.

Chacun de ces contrôles **se tait dès qu'il ne peut pas conclure** : une collection écrite sans paramètre de type (`List` nu) ne se voit jamais reprocher sa cible, puisque c'est exactement le manque que `target` vient combler. Un avertissement qui se trompe coûte plus cher que celui qui manque un cas : le premier apprend à ne plus les lire. Deux exemptions en découlent : les

composants d'un record échappent à tous les contrôles qui disent « ce membre n'est pas affiché », puisqu'un composant est dessiné quelle que soit la stratégie ; et un `@UMLInclude` qui déroge à un `hide` du projet n'est pas « sans effet », il en est le seul remède.

17.5 Faire taire les avertissements

L'option `-Aumlg.warnings=none` les éteint tous. Les erreurs ne s'éteignent pas : elles disent qu'aucun diagramme ne peut être produit. `none` est la seule valeur reconnue, et toute autre est **refusée** — `NONE`, `off`, `false` laissaient passer les avertissements sans signaler la faute de frappe. Le silence porte enfin sur ce que le processeur a trouvé dans le code, non sur son propre état : un `-Aumlg.rules` désignant un fichier introuvable reste signalé même sous `none`.

17.6 Donner le fichier de règles au compilateur

Six avertissements dépendent de la stratégie appliquée à la classe : ceux qui disent « ce membre n'est pas affiché » ou « il l'est déjà ». Quand la classe déclare sa stratégie, le processeur conclut seul. Quand elle s'en remet au projet — le défaut `PROJECT` —, la réponse est dans `umlg.rules`, un fichier que `javac` ne sait pas trouver : il vit à la racine des classes compilées, et rien, dans une compilation, ne dit ce que la construction fera de leur résultat.

Le processeur **se tait** alors sur ces six contrôles, plutôt que de parler au hasard. C'est la seule chose que la bibliothèque taise, et une option suffit à la lui rendre :

```
<compilerArgs>
    <arg>-Aumlg.rules=src/main/resources/umlg.rules</arg>
</compilerArgs>
```

Trois situations, trois gravités : **option absente**, le cas ordinaire — le processeur se tait sur ce qu'il ne peut pas savoir ; **fichier introuvable — avertissement**, le code reste juste mais son auteur attend des vérifications qu'il n'aura pas ; **fichier fautif — erreur**, le même fichier arrêterait la génération pour la même raison, et le dire ici le dit plus tôt. Cette option ne change jamais le diagramme produit, et n'ajoute ni ne retire aucune *erreur*.

18 Choix de conception : ce qui est hors périmètre

Certaines possibilités de PlantUML ont été délibérément écartées. Les mentionner évite de les réintroduire par inadvertance.

18.1 Fonctionnalités écartées

- **Style fin des liens** : couleur, épaisseur, pointillés, longueur des flèches. Gain purement esthétique pour un coût élevé en surface d'API.
- **Navigabilité explicite** (`--` contre `-->` contre `<-->`) : la flèche déduite convient dans la quasi-totalité des cas.
- **Pastille de stéréotype** (`«(S,#FF7700) Service»`) et stéréotypes multiples : `stereotype` reste une chaîne simple.
- **Association qualifiée** — le petit carré à clé qu'UML dessine à la source —, qu'un paramètre `qualifier` a un temps permis de déclarer : PlantUML n'a pas cette notation, et les crochets qui la rendaient ne sont reconnus par aucun lecteur. La donnée qui indexe se dit par le type de l'attribut, une `Map<String, Commande>` que le diagramme écrit en entier.
- **Imbrication de paquetages** et variantes de conteneurs (`namespace`, `folder`, `frame`, `together`) : un seul niveau de regroupement suffit.
- **Notes rattachées à un lien** et notes flottantes non ancrées.

— **Directives brutes** `skinparam` et thèmes PlantUML : les exposer reviendrait à réécrire PlantUML en syntaxe Java.
— **Le placement d'une boîte par rapport à une autre**, qu'une annotation `@UMLLayout` a un temps permis de forcer. Ce qu'elle apportait vraiment — ordonner des boîtes que rien ne relie — est désormais déduit de l'ordre d'écriture d'une vue (section 16), là où une carte de placements écrite à la main périmait au premier ajout de classe.
— **Boîtes, attributs et liens déclarés hors du code**, qu'une famille de règles `declare` a un temps permis d'écrire. Le diagramme ne montre que ce que le code porte : une boîte déclarée ailleurs échappait à tout ce qui fait la valeur de la bibliothèque — aucun compilateur ne la vérifiait, aucun renommage ne la suivait. Une classe encore à écrire s'écrit vide, en Java, avec `@UMLClass` : elle coûte trois lignes, et le compilateur la suit.

18.2 Ce qui relève de l'analyse automatique

Sont déduits du code compilé, sans aucune annotation : la nature de l'entité (classe, classe abstraite, interface, énumération, type annotation, *record*), les liens d'héritage lus dans `extends` et `implements`, les littéraux d'une énumération, le modificateur `static`, la visibilité de chaque membre, et les signatures de méthodes (type de retour, types des paramètres, et leurs noms sous `-parameters`).

Deux s'y ajoutent, qui touchent à la notation UML. Les **paramètres de type** d'une classe générique sont écrits à la suite de son nom — `Boite<T>`, `Table<K, V extends Number>` — avec leurs bornes, qui passent par les règles de nommage comme n'importe quel type. Ils restent distincts du nom, qui désigne la boîte : les y coller rendrait `view "X" = Boite` introuvable dès que `Boite` devient générique. Une **opération sans corps** — `abstract`, ou déclarée par une interface sans être `default` — est écrite en italique, comme UML l'exige : c'est la seule chose qui distingue à l'œil un contrat d'une implémentation.

18.3 Ce qui ne peut pas être déduit

La lecture ayant lieu après compilation (section 1.1), deux informations ont disparu : les **valeurs d'initialisation des attributs**, et le **corps des méthodes** — d'où `@UMLDependency`, seule façon de déclarer un lien qui ne se lit que dans un appel. Le **contenu des collections**, longtemps rangé ici, n'y a pas sa place : les paramètres de type survivent dans les signatures déclarées, d'où `target` facultatif sur `@UMLAssociation`. Ce qui reste réellement indéductible est la **multiplicité** : rien dans `Set<String>` ne dit `0..*` plutôt que `1..*`, cela relève du métier, et le deviner produirait un diagramme faux une fois sur deux. Quant au **nom des paramètres de méthode**, il ne s'agit pas d'une déduction impossible mais d'une option de construction du projet analysé, `-parameters` (section 2.2) : aucune annotation ne vient combler ce manque, le compilateur sachant déjà le faire.